

```

1  # Note 18: DBSCAN
2  # =====
3
4  # Import Libraries
5  import numpy as np
6  import pandas as pd
7  import matplotlib.pyplot as plt
8  from sklearn.datasets import make_moons
9  from sklearn.cluster import DBSCAN
10 from sklearn.preprocessing import StandardScaler
11 from sklearn.metrics import silhouette_score
12
13 # Header
14 print("\n=====")
15 print("DBSCAN Clustering")
16 print("=====\n")
17
18 # DBSCAN Clustering [Used Grid Search for Find Best Eps and Min Samples]
19 DBScan = DBSCAN(eps=0.7, min_samples=9)
20 DBScan.fit(X_pca)
21 labels = DBScan.labels_
22
23 print("01. Number of Clusters      : ", len(set(labels)) - (1 if -1 in labels else 0))
24 print("02. Number of Noise Points  : ", list(labels).count(-1))
25
26 # Calculate S Score
27 n_clusters = len(set(labels)) - (1 if -1 in DBScan.labels_ else 0)
28 BEST_S_SCORE_DBSCAN = 0
29 if n_clusters >= 2:
30     dbscan_score = silhouette_score(X_pca, DBScan.labels_)
31     print("03. DBSCAN Silhouette Score : ", round(dbscan_score, 2))
32     BEST_S_SCORE_DBSCAN = round(dbscan_score, 2)
33 else:
34     print("Silhouette Score cannot be calculated.")
35
36 # Plot DBSCAN
37 plt.figure()
38 plt.scatter(X_pca[:,0], X_pca[:,1], c=labels, cmap='viridis',s=50)
39
40 plt.xlabel("PCA 1")
41 plt.ylabel("PCA 2")
42 plt.title("DBSCAN Clustering For Wine Clustering Dataset")
43 plt.grid(True)
44
45 plt.show()
46
47 # Plot the Comparison
48 plt.figure()
49 X = ['DBSCAN', 'KMeans']
50 Y = [BEST_S_SCORE_DBSCAN, BEST_S_SCORE_KMEANS]
51 plt.scatter(X, Y, color='red', marker='o', s=10)
52 plt.plot(X, Y, color='blue', linestyle='--')
53 plt.xlabel("Models")
54 plt.ylabel("Silhouette Score")
55 plt.title("Comparison of KMeans and DBSCAN For Wine Clustering Dataset")
56 plt.grid(True, alpha=0.3)
57
58 # Add value labels
59 for i in range(len(X)):
60     plt.text(
61         X[i],                # x-coordinate
62         Y[i] + 0.001,        # slightly above the point
63         f"{Y[i]:.2f}",        # show 3 decimal places
64         ha='center'
65     )
66
67 plt.show()
68
69 # Number of Clusters
70 n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
71 print("\nNumber of clusters:", n_clusters)
72

```

```
73 # Number of Noise Points
74 n_noise = list(labels).count(-1)
75 print("Number of noise points:", n_noise)
76
77 # Visualize DBSCAN clusters
78 # =====
79
80 plt.figure(figsize=(8, 5))
81
82 plt.scatter(
83     X[:, 0],
84     X[:, 1],
85     c=labels,
86     cmap="viridis"
87 )
88
89 plt.xlabel("Feature 1")
90 plt.ylabel("Feature 2")
91 plt.title("DBSCAN Clustering")
92
93 plt.show()
```